Guia de integração - Llama Desktop Launcher

Versão 0.7.0 · Português (Brasil)

Este guia explica como conectar clientes e aplicativos ao servidor iniciado pelo launcher: endpoint, ID do modelo, chaves, configurações prontas para OpenCode e outros clientes, exemplos de código, rede e firewall. Para instalar e usar o aplicativo, veja o **Manual de uso** (MANUAL-pt-BR). Para resolver problemas, veja o **Guia de diagnóstico** (DIAGNOSTICS-pt-BR).

1. Visão geral

Os três motores (llama.cpp, vLLM e Strata/PolyStrata) expõem uma API compatível com **OpenAI Chat Completions** em /v1. O launcher não cria uma API pública na internet: o servidor só existe enquanto o launcher está aberto e com o servidor iniciado, e só atende o endereço escolhido.

Há três formas de acesso:

Modalidade	Endereço	Credencial	Validade
Direta local	http://127.0.0.1:PORTA/v1	Nenhuma (use local em clientes que exigem o campo)	Enquanto o servidor estiver ativo
Direta em rede	http://IP-LAN-OU-TAILSCALE:PORTA/v1	Chave permanente da API	A chave vale entre reinícios até você gerar outra
Compartilhada	http://IP-LAN-OU-TAILSCALE:PORTA-COMPARTILHADA/v1	Credencial temporária de compartilhamento	Até revogar, parar o servidor ou fechar o launcher

Não misture as credenciais: a chave permanente só vale na porta direta, e a credencial temporária só vale na porta compartilhada.

2. Endpoint e portas

Escolha o **Endereço de acesso** antes de clicar em **Iniciar servidor**. Depois que o modelo carrega, o painel **Conectar aplicativo** mostra uma linha como esta:

API: http://192.168.1.5:8080/v1 · Modelo API: modelo-local · Interface: http://192.168.1.5:8080/

- **API direta:** a primeira porta livre entre **8080 e 8180** no endereço escolhido. **Copiar link da API** copia o endereço terminado em /v1.
- **API compartilhada:** com LAN ou Tailscale, uma segunda porta entre **8181 e 8280**, mostrada como **API compartilhada**.
- **Ponte local:** quando você clica em **Abrir no navegador** com acesso em rede, o launcher abre a interface por uma porta local de 127.0.0.1 que já autentica. Ela só serve para este computador.

As portas são exemplos: elas mudam se outra estiver ocupada. Depois de reiniciar o servidor ou o computador, confira o IP e a porta e atualize os clientes. Em outro computador, 127.0.0.1 aponta para o próprio cliente, não para o servidor.

O launcher detecta como **Rede interna** os endereços 10.0.0.0/8, 172.16.0.0/12 e 192.168.0.0/16, e como **Tailscale** os endereços 100.64.0.0/10. Endereços públicos e IPv6 não aparecem na lista.

3. ID do modelo por motor

O cliente precisa mandar o campo model. Use o **Modelo API** mostrado no painel:

Motor	ID do modelo	Observação
llama.cpp	modelo-local	Fixo, para que os clientes não precisem mudar ao trocar de GGUF.
vLLM	modelo-local	O launcher passa --served-model-name modelo-local.
Strata / PolyStrata	model_name da configuração strata-*.json	Sem model_name, é o nome do arquivo sem strata- e .json.

As configurações copiadas pelo launcher já trazem o ID certo. Para trocar de modelo, pare o servidor, selecione outro e inicie de novo. Clientes já abertos não reiniciam a conversa sozinhos, e o limite de contexto pode ter mudado.

Confira o ID com GET /v1/models (exemplos na seção 7).

4. Chaves e credenciais

Chave permanente da API

- É criada no primeiro início em rede e fica em server-api-key.txt, na pasta de dados do aplicativo.
- **Mostrar chave** exibe a chave, e **Copiar chave** a copia para a área de transferência.
- **Nova chave** gera outra e invalida a anterior. Pare o servidor antes e atualize os clientes de acesso direto.
- O cliente envia Authorization: Bearer SUA_CHAVE. O link sozinho não dá acesso.
- Guarde a chave no gerenciador de segredos do cliente ou em uma variável de ambiente, fora do código e do repositório. Não coloque a chave na URL nem em mensagens abertas.

Credencial temporária de compartilhamento

- É gerada a cada início do servidor em LAN ou Tailscale e vai embutida em **Copiar link com acesso**, **Copiar Bash/PowerShell com acesso** e **Copiar configuração com acesso**.
- **Revogar acesso compartilhado** troca a credencial e derruba as conexões compartilhadas. Copie de novo para dar acesso outra vez.
- Parar o servidor ou fechar o launcher encerra o compartilhamento; ao reiniciar, a credencial é outra.
- No navegador, o token sai da barra de endereço depois da autenticação e fica em um cookie HttpOnly.
- Revogar o compartilhamento **não** revoga a chave permanente nem os clientes que a usam.

5. Clientes prontos

Com o servidor ativo, escolha o cliente em **Conectar aplicativo**. O launcher não instala clientes nem sobrescreve configurações: ele copia um comando ou um trecho de configuração para você colar.

- **Comandos** (OpenCode e Aider): **Copiar comando Bash** (Linux/macOS) ou **Copiar comando PowerShell** (Windows). Cole no terminal, dentro da pasta do projeto. Em rede, o comando pede a chave com entrada oculta; a chave não vai no texto copiado.

- **Configuração** (todos): **Copiar configuração**. Na rede, substitua SUBSTITUA_PELA_CHAVE_DA_API pela chave de **Copiar chave**. A versão **com acesso** já traz a credencial temporária e o endpoint compartilhado.

As configurações usam como contexto por sessão o contexto do servidor dividido pelas sessões simultâneas, e como limite de saída o menor entre 1024 e metade desse contexto. Copie de novo se mudar o modelo, o endereço, o contexto ou as sessões.

Cliente	Como usar a configuração copiada
OpenCode	O comando define o provedor llama-local só para aquela execução via OPENCODE_CONFIG_CONTENT e roda opencode --model llama-local/ID. Para deixar fixo, mescle a configuração copiada no opencode.json do projeto ou em ~/.config/opencode/opencode.json.
Aider	O comando define OPENAI_API_BASE e OPENAI_API_KEY só para aquela execução e roda aider --model openai/ID.
Pi	Mescle em ~/.pi/agent/models.json, preservando os providers existentes, e selecione o modelo do provider llama-local em /model.
Hermes	Mescle o bloco model no config.yaml e rode hermes chat. Versões atuais exigem pelo menos 64000 tokens por sessão para usar ferramentas; a configuração exporta o contexto real, sem aumentá-lo.
Continue	Mescle o bloco models no config.yaml, preservando os modelos existentes.
Cline	Em configurações, escolha **OpenAI Compatible** e preencha Base URL, API Key e Model ID.
Codex	Experimental: exige Responses API, que não foi validada com estes motores. Mescle em ~/.codex/config.toml e defina LLAMA_API_KEY.
Genérico	Campos Base URL, API Key, Model ID, janela de contexto e limite de saída para qualquer cliente OpenAI Compatible.

O cliente edita os arquivos da máquina onde ele roda; a inferência acontece no servidor. Ferramentas (tool calling), visão e qualidade dependem do modelo e do template dele. Não ative recursos só porque o endpoint se declara compatível com OpenAI.

6. Integrar o seu aplicativo

Use **Genérico - OpenAI Compatible** para um aplicativo próprio, serviço, automação ou qualquer cliente que aceite Chat Completions.

1. Escolha o endereço antes de iniciar: **Somente este computador** para um cliente no mesmo PC, **Rede interna** para aparelhos da mesma rede, **Tailscale** para aparelhos da sua tailnet.
2. Clique em **Iniciar servidor** e espere aparecer o endpoint. Em LAN/Tailscale, libere o firewall (seção 9).
3. Escolha a modalidade (seção 1) e copie a configuração correspondente.
4. Configure **Base URL** com o endereço terminado em /v1, **API Key** com a credencial e **Model ID** com o ID do painel. Se o cliente já acrescenta /chat/completions, não repita esse caminho.
5. Teste primeiro GET /v1/models e depois POST /v1/chat/completions com JSON e o cabeçalho Authorization: Bearer CHAVE.

A janela de contexto de cada pedido é o contexto do servidor dividido pelas sessões simultâneas. Prompt, histórico, ferramentas e resposta precisam caber nela. A API só devolve texto: executar código ou editar arquivos é responsabilidade do seu aplicativo.

7. Exemplos

Nos exemplos, a Base URL já termina em /v1, por isso acrescentamos só /models ou /chat/completions. Troque modelo-local pelo ID do Strata quando for o caso. Para acesso direto local, informe a chave local.

Linux/macOS com Bash e curl

A chave é pedida sem aparecer na tela.

```
(
  read -rp 'Base URL (terminada em /v1): ' LLAMA_BASE_URL
  read -rsp 'API Key: ' LLAMA_API_KEY
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    "${LLAMA_BASE_URL%/}/models"
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    -H 'Content-Type: application/json' \
    "${LLAMA_BASE_URL%/}/chat/completions" \
    --data '{"model":"modelo-local","messages":[{"role":"user",
    "content":"Responda apenas: conectado"}],"max_tokens":64,
    "stream":false}'
)
```

O primeiro resultado lista o ID do modelo. No segundo, a resposta fica em choices[0].message.content.

Windows com PowerShell

```
& {
  $baseUrl = (Read-Host 'Base URL (terminada em /v1)').TrimEnd('/')
  $secureKey = Read-Host 'API Key' -AsSecureString
  $apiKey = [System.Net.NetworkCredential]::new('', $secureKey).Password
  $headers = @{ Authorization = "Bearer $apiKey" }
  Invoke-RestMethod -Uri "$baseUrl/models" -Headers $headers
  $body = @{
    model = 'modelo-local'
    messages = @(@{ role = 'user'; content = 'Responda: conectado' })
    max_tokens = 64
    stream = $false
  } | ConvertTo-Json -Depth 5
  $response = Invoke-RestMethod -Method Post `
    -Uri "$baseUrl/chat/completions" -Headers $headers `
    -ContentType 'application/json; charset=utf-8' `
    -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
  $response.choices[0].message.content
}
```

Python (biblioteca padrão)

Defina LLAMA_BASE_URL, LLAMA_API_KEY e, se preciso, LLAMA_MODEL no ambiente do processo.

```
import json
import os
from urllib.request import Request, urlopen

base_url = os.environ['LLAMA_BASE_URL'].rstrip('/')
api_key = os.environ['LLAMA_API_KEY']
payload = {
```

```
    'model': os.environ.get('LLAMA_MODEL', 'modelo-local'),
    'messages': [{'role': 'user', 'content': 'Responda: conectado'}],
    'max_tokens': 64,
    'stream': False,
}
request = Request(
    base_url + '/chat/completions',
    data=json.dumps(payload).encode('utf-8'),
    headers={
        'Authorization': 'Bearer ' + api_key,
        'Content-Type': 'application/json',
    },
    method='POST',
)
with urlopen(request, timeout=120) as response:
    result = json.load(response)
print(result['choices'][0]['message']['content'])
```

O timeout de 120 segundos é um exemplo; modelos grandes podem precisar de mais.

Streaming

Com "stream": true, a resposta chega em Server-Sent Events (SSE): linhas data: {...} com pedaços em choices[0].delta.content, terminando em data: [DONE]. Use uma biblioteca que trate SSE ou leia linha a linha. Os exemplos acima usam resposta completa para ficar simples.

8. Rede: LAN ou Tailscale

- **Tailscale** é a opção preferida: o tráfego é cifrado e só aparelhos autorizados na sua tailnet chegam ao servidor. O cliente precisa estar na mesma tailnet.
- **Rede interna (LAN)** usa HTTP sem cifra. Use só em redes confiáveis; qualquer um na rede pode ver o conteúdo trafegado, embora precise da chave para usar o modelo.
- O endereço LAN pode mudar quando o roteador renova o DHCP. Se for usar sempre, reserve o IP no roteador.

9. Firewall

Windows

O launcher não mexe no firewall ao iniciar o servidor. Com o servidor ativo em LAN ou Tailscale, a seção **Firewall do Windows** da tela **Llama** cria as regras:

1. Confira a **faixa de clientes** sugerida (por exemplo 192.168.1.0/24 ou 100.64.0.0/10). Ela precisa conter o endereço escolhido e ficar dentro de uma rede privada ou Tailscale.
2. Clique em **Criar regra para a porta ativa** e confirme o pedido de administrador do Windows.
3. O launcher cria uma regra de entrada TCP para a porta da API e outra para a porta compartilhada, cada uma limitada ao executável, à porta e à faixa indicada.

Motor	Executável liberado na porta da API
llama.cpp	llama-server.exe da pasta do motor
Strata / PolyStrata	O python.exe base do ambiente .venv (lido de pyvenv.cfg)
vLLM (Docker)	Nenhum: a porta é publicada pelo Docker Desktop, e a regra precisa ser manual

A porta compartilhada é liberada para o executável do launcher. Se a porta mudar, crie outra regra. As regras se chamam LlamaDesktopLauncher-… e aparecem no **Windows Defender Firewall com Segurança Avançada**, onde você pode apagá-las.

Para o vLLM no Docker, o Docker Desktop costuma criar a própria regra de entrada na instalação. Se os clientes não conectarem, crie uma regra para a porta mostrada, limitada à faixa, em um PowerShell como administrador:

```
New-NetFirewallRule -DisplayName 'vLLM Docker 8080' -Direction Inbound `
  -Action Allow -Protocol TCP -LocalPort 8080 -RemoteAddress 192.168.1.0/24
```

Linux

Libere a porta da API e a compartilhada só para a faixa dos clientes. Exemplos para a porta 8080:

```
sudo ufw allow from 192.168.1.0/24 to any port 8080 proto tcp
sudo firewall-cmd --add-rich-rule='rule family="ipv4" source address="192.168.1.0/24" port port="8080" protocol="tcp" acce
```

Use o comando da sua distribuição (ufw no Ubuntu e derivados, firewalld no Fedora e RHEL). A regra do firewall-cmd acima vale até reiniciar; acrescente --permanent e rode sudo firewall-cmd --reload para mantê-la.

macOS

Se o firewall do macOS estiver ligado, o sistema pergunta se o llama-server pode aceitar conexões na primeira vez que ele escuta na rede. Permita só se for usar acesso em rede.

10. Verificação rápida

Resultado	O que conferir
Conexão recusada ou timeout	Servidor iniciado e modelo carregado, IP e porta atuais, mesma rede ou Tailscale ativo, firewall.
401 ou 403	Cabeçalho Bearer presente, credencial certa para a porta (permanente na direta, temporária na compartilhada), compartilhamento não revogado.
404	Base URL terminando em /v1, sem /v1/v1; cliente usando Chat Completions.
400 ou erro de contexto	JSON válido, ID do modelo certo, pedido dentro do contexto por sessão.
Cliente não reconhece o modelo novo	Confira /v1/models, reinicie a conversa e atualize o ID, o limite e a credencial.

O **Guia de diagnóstico** detalha cada caso e as mensagens do aplicativo.